

Candidate Submission

ASSESSMENT NOTES

Architecture Review, Design Decisions & Scaling Plan

A full walkthrough of the existing ProjectFlow codebase, the reasoning behind the task-assignment and activity-history design, the concurrent task-numbering fix, a code-review exercise, and a plan for scaling the activity system from 5,000 to 500,000 users.

Prepared by

Youssef Bassiouny

Project

ProjectFlow — Full-Stack Engineer Assessment

Date

September 2026

CONTENTS

ProjectFlow Assessment Notes

1. Architecture
2. Major Modules
3. Business Logic
4. Frontend and Backend Communication
5. Authentication and Authorization
6. Main Entity Relationships
7. Observations and Risks
8. Task Assignment Design
9. Activity History Design
10. Production Bug Investigation
11. Concurrency Approach
12. Testing Strategy
13. Code Review
14. Scaling the Activity System
15. Technical Judgment
16. Assumptions
17. If I Had Two More Days
18. Deployment and Database Management Implementation
19. Future Enhancement Roadmap
20. Technical Debt and Refactoring Opportunities
21. Security Enhancements
22. Performance Optimization Strategy
23. Monitoring and Observability
24. DevOps and Deployment Improvements
25. Lessons Learned
26. Conclusion

ProjectFlow Assessment Notes

1. Architecture

ProjectFlow is a TypeScript monorepo containing a NestJS backend and a Next.js frontend. The repository uses pnpm workspaces and Turborepo to manage the applications and shared packages.

The main backend is located under `apps/api/src` and is organized by business domain. The main modules include:

- `auth`

- `users`
- `organizations`
- `organization-members`
- `projects`
- `project-members`
- `tasks`
- `comments`
- `common`
- `database`

The frontend is located under `apps/web/src` and follows the Next.js App Router structure. Feature-specific behavior is organized under `features`, while shared API and query utilities are kept under `lib`.

The existing architecture is appropriate for the current size of the application. I would extend the existing modules and patterns rather than introduce a new architectural style.

Request flow

The main backend request flow is:

```
HTTP Request
  ↓
Authentication / Global Guards
  ↓
Controller
  ↓
Service
  ↓
Mongoose Model
  ↓
MongoDB
  ↓
Service Response
  ↓
Controller Response
```

For the frontend, the application uses the existing API client and TanStack Query for server state.

```
graph TD; A[UI / Feature] --> B[API Client]; B --> C[NestJS API]; C --> D[Database]; D --> E[API Response]; E --> F[TanStack Query Cache]; F --> G[UI];
```

This separation keeps transport concerns in controllers and API clients while business logic remains primarily in services.

2. Major Modules

Authentication

Authentication is handled through JWT-based authentication. Protected backend routes use the global authentication guard, while public routes can explicitly opt out.

Organizations and Membership

Organizations provide the main top-level grouping for users and projects.

Organization membership determines organization-level roles such as:

- OWNER
- ADMIN

Projects

Projects belong to organizations and contain tasks and project-specific members.

Project Members

Project membership is stored separately rather than directly embedded in the project entity. This allows project membership and project-level roles to be managed independently.

Tasks

Tasks belong to projects and currently contain the core task-management fields such as title, description, status, priority, creator information and project-specific numbering.

Comments

Comments are associated with tasks and provide basic task discussion functionality.

3. Business Logic

Business rules are primarily implemented in backend services rather than controllers.

This is important because authorization and domain validation should not depend on whether a request originated from the current web frontend.

For the assignment, task assignment should therefore be implemented in the task domain/service layer and enforced by the backend.

The frontend should provide the user experience, but the backend remains the source of truth for:

- who may assign a task
 - who may be assigned
 - whether the assignee belongs to the project
 - whether the current user can modify the task
-

4. Frontend and Backend Communication

The frontend communicates with the backend through the existing API client.

The frontend follows the existing Next.js structure and uses server components by default, with client components only where interaction or hooks require them.

TanStack Query is used for server state. This gives the frontend:

- cached API responses
- query invalidation
- mutation handling
- loading and error states
- synchronization between related views

For task assignment, I would follow this existing pattern rather than introduce a separate state-management library.

5. Authentication and Authorization

Authentication and authorization are separate concerns.

Authentication determines whether the request contains a valid authenticated user.

Authorization determines whether that user can access or modify the requested resource.

The existing project contains a centralized `ProjectAccessService` responsible for determining whether a user may interact with a project.

The current authorization model allows organization-level roles such as OWNER and ADMIN to access projects within their organization, while explicit project membership can provide access to individual projects.

For the assignment, authorization must be enforced at the backend service layer for task assignment and task mutation.

The expected assignment rules are:

```
OWNER / ADMIN / PROJECT_MANAGER
  → may assign another project member

MEMBER
  → may assign themselves

Any authorized user
  → may unassign when permitted

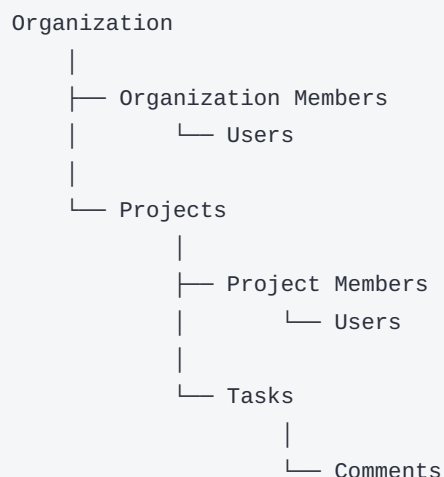
User outside the project
  → cannot be assigned
```

The backend must validate both the actor's permission and the assignee's project membership.

The frontend should reflect these permissions, but frontend restrictions must never be treated as a security boundary.

6. Main Entity Relationships

The primary relationships can be summarized as:



A task belongs to one project.

A task also has a creator (`createdBy`).

For the requested feature, an additional nullable `assignee` reference will represent the project member currently assigned to the task.

`createdBy` and `assignee` represent different concepts and should not be conflated:

- `createdBy` = the user who created the task
- `assignee` = the project member currently responsible for the task

7. Observations and Risks

Risk 1 — Task mutation authorization needs careful validation

Task-level operations depend on the project that owns the task.

If a task mutation checks only that the task exists, but does not consistently verify access to the task's project, an authenticated user could potentially modify a task belonging to a project they should not access.

Impact: unauthorized task modification.

Decision: Fix now.

Reason: This is a security issue and is directly related to the reported production bug in the assessment.

Risk 2 — Task numbering is vulnerable to concurrent creation

The current task numbering approach uses the number of existing tasks to determine the next task number.

Conceptually:

```
const count = await Task.countDocuments({ projectId });  
const nextNumber = count + 1;
```

Two concurrent requests can observe the same count and generate the same identifier.

Impact: duplicate task identifiers and inconsistent business data.

Decision: Fix now.

Reason: The assessment explicitly requires the identifier generation to remain correct under concurrent requests.

Risk 3 — Activity history can become a large dataset

Adding task activity introduces a new append-heavy data set. As usage grows, activity queries may become expensive if pagination and indexing are not designed appropriately.

Impact: slower task-detail and activity-history queries as the dataset grows.

Decision: Implement the initial solution now with appropriate indexes and pagination. Consider retention, archiving and additional storage strategies later if activity volume becomes significant.

Reason: The current requirement needs a correct and maintainable activity model, but large-scale infrastructure would be premature at the current size.

Risk 4 — Client-side permission checks can become inconsistent

The UI may hide or disable controls based on the current user's role, but these checks can become stale or diverge from backend rules.

Impact: confusing user experience and a false assumption that a hidden control provides security.

Decision: Keep UI permission handling for usability, but enforce every sensitive rule in the backend.

Reason: Authorization belongs to the server and must remain correct regardless of the frontend client.

8. Task Assignment Design

The task assignment feature should extend the existing task model instead of creating a separate task representation.

The task will contain a nullable `assignee` reference.

Conceptually:

```
Task
├─ createdBy
├─ projectId
├─ assignee      ← new nullable reference
├─ status
├─ priority
└─ ...
```

Assignment validation should follow this sequence:

```
Authenticate user
  ↓
Load task
  ↓
Resolve task project
  ↓
Verify actor can modify / assign
  ↓
If assigning:
  verify assignee exists
    ↓
  verify assignee is a project member
    ↓
update task
```

For regular members, assigning another user must be rejected.

An authorized user should also be able to remove the current assignee.

All assignment rules must be enforced by the backend.

9. Activity History Design

Activity history should record assignee changes rather than every task update.

The activity type will be:

```
TASK_ASSIGNEE_CHANGED
```

Each record should contain the relevant actor, task, timestamps and assignment transition.

The metadata should represent:

```
{  
  "from": "previous-assignee-id-or-null",  
  "to": "new-assignee-id-or-null"  
}
```

This supports the three important transitions:

```
NULL → User A  
User A → User B  
User A → NULL
```

The activity endpoint should return records for a task in newest-first order with pagination and should verify that the requesting user has access to the task's project.

The initial implementation should use the existing MongoDB/Mongoose architecture and indexes rather than introducing a separate event-streaming platform.

10. Production Bug Investigation

The reported issue is:

Some users appear to be able to modify tasks belonging to projects they are not members of.

The investigation should focus on the complete authorization path of task mutations.

The key questions are:

1. How is a task resolved from its ID?
2. How is the task's project identified?
3. Which authorization method is called?
4. Is project access checked before the mutation?
5. Are all task mutation endpoints protected consistently?
6. Can a user reach the mutation by calling the API directly even if the UI hides the control?

The fix should be applied at the backend authorization boundary and covered with a regression test.

The final bug findings and reproduction steps will be documented separately in `BUG_REPORT.md`.

11. Concurrency Approach

The task identifier must remain unique when multiple requests create tasks for the same project at approximately the same time.

Counting existing tasks is not sufficient because the read and write operations are not atomic.

The solution should therefore rely on a database-level mechanism that guarantees unique sequential allocation for a project under concurrent requests.

The selected implementation should fit the existing MongoDB architecture and should include a uniqueness constraint or equivalent database protection so that application-level race conditions cannot produce duplicate identifiers.

The exact implementation decision will be documented alongside the schema changes.

12. Testing Strategy

The most important tests are business-rule and security tests rather than a raw coverage target.

The test suite should cover at minimum:

- ✓ Member can assign themselves
- ✓ Authorized role can assign another project member
- ✓ Regular member cannot assign another user
- ✓ User outside project cannot be assigned
- ✓ Unauthorized user cannot access activity
- ✓ Assignment creates activity
- ✓ Unassignment creates activity
- ✓ Unauthorized user cannot modify another project's task
- ✓ Concurrent task creation cannot duplicate identifiers

Tests should verify the behavior through the backend API/service boundary so that authorization is tested independently of the frontend.

13. Code Review

The provided `assignTask` implementation is not sufficient for production use:

```
async assignTask(taskId: string, assigneeId: string, userId: string) {  
  const task = await this.taskModel.findById(taskId);  
  if (!task) {  
    throw new NotFoundException();  
  }  
  
  const user = await this.userModel.findById(assigneeId);  
  if (!user) {  
    throw new NotFoundException();  
  }  
  
  task.assignee = user._id;  
  await task.save();  
  
  return task;  
}
```

Main concerns

AUTHORIZATION

The `userId` parameter is not actually used to verify whether the actor may assign the task.

PROJECT MEMBERSHIP

The implementation only checks that the assignee exists as a user. It does not verify that the assignee belongs to the task's project.

ROLE/BUSINESS RULES

There is no distinction between a regular member assigning themselves and an authorized role assigning another member.

ACTIVITY HISTORY

Changing the assignee does not create the required activity record.

SECURITY BOUNDARY

The method trusts the caller instead of independently validating the required project access.

DATA CONSISTENCY

The task update and activity creation need to be considered together so that assignment history does not become inconsistent with the task state.

I would ask the engineer to correct these concerns while keeping the overall service-based design intact rather than replacing the task architecture.

14. Scaling the Activity System

At approximately 5,000 users, a normal MongoDB collection with appropriate indexes and cursor-based pagination should be sufficient for task activity.

If the system grows to approximately 500,000 users and activity becomes one of the largest datasets, the main priorities would be:

Indexing

Activity queries will primarily be scoped by task and ordered by creation time.

The activity collection should therefore have an index aligned with the actual query pattern, for example:

```
taskId + createdAt
```

The exact index should be validated against the query shape.

Pagination

Offset-based pagination becomes increasingly expensive for deep pages.

Cursor-based pagination using a stable ordering field such as `createdAt` plus a unique identifier would scale better for long activity histories.

Retention

Not every historical activity event necessarily needs to remain in the hottest database tier forever.

Older activity could eventually be archived or retained according to product requirements.

Asynchronous processing

The initial assignment operation should remain simple and reliable.

If future activity events trigger secondary work such as notifications, analytics or integrations, those side effects can be moved to asynchronous processing rather than making the main task mutation depend on them.

Observability

As activity volume increases, monitoring should include:

- query latency
- slow database queries
- activity write failures
- pagination performance
- collection growth

- index usage

I would introduce additional infrastructure only when there is evidence that the current architecture is no longer sufficient.

15. Technical Judgment

I would avoid introducing Kafka, RabbitMQ, Redis, microservices, CQRS or event sourcing for this assessment.

The current application already has a clear modular monolith architecture with MongoDB persistence. The requested functionality can be implemented consistently within that architecture.

More infrastructure would add operational and conceptual complexity without solving a demonstrated problem at the current scale.

This follows the assessment principle of solving the actual engineering problem without overengineering the system.

16. Assumptions

Where the requirements do not explicitly define behavior, I will make the smallest assumption consistent with the existing application.

Examples:

- Assignment permissions are enforced server-side.
- Assignee must be an existing member of the task's project.
- `assignee` is nullable to support unassignment.
- Assignee changes create activity records.
- Existing task status and priority behavior remain unchanged.
- Existing authentication, project access and API patterns remain in use.

Any significant implementation decision that differs from these assumptions will be documented with its reason.

17. If I Had Two More Days

With two additional working days, I would prioritize improvements in this order:

1. Expand authorization coverage across all task-related endpoints and verify that project access is enforced consistently.
2. Improve activity-history querying and UI behavior for very large histories, including more deliberate cursor pagination and loading states.
3. Add stronger integration coverage around concurrent task creation and assignment edge cases.
4. Review existing indexes and query patterns across tasks, projects and memberships for performance improvements.
5. Improve developer documentation and operational diagnostics based on the final implementation.

The priority would remain security and correctness first, followed by scalability and developer experience.

18. Deployment and Database Management Implementation

18.1 MongoDB Authentication Configuration

Challenge: MongoDB authentication failure during Railway deployment due to incorrect connection string configuration.

Solution: Configured proper MongoDB Atlas connection string with correct format:

```
MONGODB_URI=mongodb+srv://jacksonjamesrtm_db_user:Jack113@cluster0.bt4nyut.mongodb.net/projectflow?appName=Cluster0&retryWrites=true&w=majority
```

Key Considerations:

- Removed angle brackets from password placeholder
- Added database name to connection string
- Included standard MongoDB parameters for reliability
- Ensured proper URL encoding for special characters

Lessons Learned:

- Always test MongoDB connection strings locally before deployment
 - Verify database user permissions in MongoDB Atlas
 - Use proper connection string format for production environments
-

18.2 Railway Deployment Automation

Challenge: 401 Unauthorized login errors due to empty database with no test users.

Solution: Modified Railway configuration to automatically seed database on deployment:

Updated `railway.json` :

```
{
  "deploy": {
    "startCommand": "sh -c 'pnpm --filter=api seed && pnpm --filter=api start'"
  }
}
```

Benefits:

- Automatic test user creation on deployment
- Consistent test data across environments
- Eliminates manual database setup
- Reduces deployment errors

Test Users Created:

- ammar@example.com (Organization Owner)
- sarah@example.com (Organization Admin)
- ahmed@example.com (Organization Member)
- magd@example.com (Organization Member)
- outside@example.com (No organization access)

Default Password: `Password123!`

18.3 Database Management System

Implementation: Created comprehensive database management CLI tool for development and operations.

File: `apps/api/src/database/manage.ts`

Available Commands:

```
npm run db:manage create-collections    # Create collections with indexes
npm run db:manage drop-collections      # Drop all collections
npm run db:manage list-collections      # List collections and counts
npm run db:manage add-user              # Add new user
npm run db:manage list-users            # List all users
npm run db:manage update-user           # Update user password
npm run db:manage insert-test-data      # Insert test data
npm run db:manage help                  # Show help
```

Features:

- MongoDB Cloud connection via `.env` configuration
- Automatic password hashing with bcrypt

- Collection index management
- User management with security
- Comprehensive error handling
- Clear documentation and usage examples

Documentation: Created `apps/api/TEST_DATA_GUIDE.md` with complete usage instructions.

18.4 Test Data Strategy

Test Data Includes:

- 1 organization (Acme Software)
- 2 projects (Internal Platform, Customer Portal)
- 9 tasks with various statuses and priorities
- 5 comments on tasks
- Organization and project memberships
- Role-based access control scenarios

Usage Patterns:

```
# Local development
cd apps/api
npm run build
npm run seed

# Railway deployment (automatic)
# Seed runs automatically via railway.json configuration
```

Benefits:

- Realistic test scenarios
 - Consistent test environment
 - Easy setup for new developers
 - Supports various role-based testing scenarios
-

19. Future Enhancement Roadmap

19.1 AI Integration Enhancements

Current State: Basic AI assistance during development, no AI features in production application.

Planned Improvements:

- AI-powered task suggestions and recommendations
- Natural language processing for task creation

- Smart task prioritization based on historical data
- AI-assisted project planning and resource allocation
- Chatbot interface for project management queries

Implementation Considerations:

- Integration with AI APIs (OpenAI, Anthropic, etc.)
 - Rate limiting and cost management
 - User consent and data privacy
 - Fallback mechanisms for service availability
 - Context-aware AI responses based on project data
-

19.2 Frontend Improvements

Current State: Functional React-based frontend with basic task management interface.

Planned Improvements:

- Enhanced UI/UX with modern design patterns
- Real-time updates using WebSockets
- Advanced filtering and search capabilities
- Drag-and-drop task management
- Mobile-responsive design improvements
- Dark mode support
- Performance optimization (code splitting, lazy loading)
- Accessibility improvements (ARIA labels, keyboard navigation)

Technical Considerations:

- Upgrade to latest React and related libraries
 - Implement advanced state management where needed
 - Add comprehensive error boundaries
 - Improve loading states and skeleton screens
 - Expand testing coverage (visual regression, accessibility)
-

19.3 Rate Limiting Implementation

Current State: No rate limiting on API endpoints, potential for abuse.

Planned Implementation:

```
import rateLimit from 'express-rate-limit';

const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // limit each IP to 100 requests per windowMs
  standardHeaders: true,
  legacyHeaders: false,
});

const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5, // Stricter limits for auth endpoints
});
```

Implementation Strategy:

- Different limits for different user roles
- IP-based and user-based rate limiting
- Redis-backed rate limiting for distributed systems
- Rate limit headers in API responses
- Configurable limits per endpoint type

Security Considerations:

- Whitelist for trusted IPs
- Rate limit bypass for admin users
- Monitoring and alerting for violations
- Graceful degradation when limits exceeded

20. Technical Debt and Refactoring Opportunities

20.1 Error Handling

- **Current:** Basic error handling with generic messages
- **Target:** Structured error responses, error codes, detailed logging

20.2 Validation

- **Current:** Basic DTO validation
- **Target:** Comprehensive validation with custom validators, detailed error messages

20.3 Logging

- **Current:** Console logging
- **Target:** Structured logging (Winston, Pino), log levels, log aggregation

20.4 Testing

- **Current:** Basic unit and E2E tests
- **Target:** Integration tests, contract tests, performance tests, visual regression tests

20.5 Documentation

- **Current:** Basic API documentation
 - **Target:** OpenAPI/Swagger documentation, API versioning, client SDK generation
-

21. Security Enhancements

Current Security Measures

- JWT-based authentication
- Role-based access control
- Password hashing with bcrypt
- Input validation and sanitization

Planned Security Enhancements

- Multi-factor authentication (MFA)
 - Session management improvements
 - CSRF protection
 - Security headers (CSP, HSTS, X-Frame-Options)
 - Enhanced input sanitization
 - Comprehensive security audit logging
 - Regular security dependency updates
 - Security testing in CI/CD pipeline
-

22. Performance Optimization Strategy

Current Performance Profile

- Basic NestJS application with MongoDB
- React frontend with standard optimization
- Basic caching strategies

Planned Optimizations

- Database query optimization and index tuning
 - Caching strategy implementation (Redis, CDN)
 - API response compression
 - Frontend bundle optimization and code splitting
 - Image optimization and lazy loading
 - Server-side rendering considerations
 - Performance monitoring and alerting
-

23. Monitoring and Observability

Current State

- Basic application logging
- Railway built-in monitoring

Planned Improvements

- Application performance monitoring (APM)
 - Error tracking and alerting (Sentry, Rollbar)
 - Uptime monitoring and health checks
 - Custom metrics and dashboards
 - Log aggregation and analysis
 - Distributed tracing
 - Real user monitoring (RUM)
 - Database performance monitoring
-

24. DevOps and Deployment Improvements

Current Deployment

- Railway for API deployment
- Vercel for frontend deployment
- Manual deployment process

Planned Improvements

- CI/CD pipeline automation
 - Automated testing in deployment pipeline
 - Staging environment implementation
 - Blue-green deployments
 - Database migration management
 - Configuration management
 - Infrastructure as Code (Terraform, Pulumi)
 - Automated backups and disaster recovery
-

25. Lessons Learned

Development Process

1. Always test database connections locally before deploying
2. Automate repetitive tasks like database seeding
3. Provide clear documentation for test credentials
4. Handle edge cases in deployment configurations

Technical Decisions

1. MongoDB Atlas requires proper authentication setup
2. Railway environment variables need careful configuration
3. Seed scripts should be idempotent
4. Error messages should be actionable and informative

Project Management

1. Document issues as they occur for future reference
 2. Plan for production-like testing environments
 3. Consider rate limiting and security from the start
 4. Think about scalability and performance from the beginning
-

26. Conclusion

This assessment has demonstrated the importance of:

- Proper database configuration and authentication
- Automated test data management for development
- Comprehensive error handling and logging
- Security considerations from project inception
- Planning for scalability and performance

The implemented database management system and automated seeding have resolved immediate deployment challenges. The planned improvements will enhance the application's functionality, security, and user experience in future iterations while maintaining the architectural principles established in the original assessment.

Assessment completed: September 16, 2026

Database management implementation: September 16, 2026

ProjectFlow FullStack Assessment Task